

SOFTWARE DEPENDABILITY COURSE PROJECT AA 2024/25

Apache Commons-Net

An In-Depth Analysis of Performance, Security, and Key
Components

PROJECT EVALUATION AND ANALYSIS OVERVIEW

The objective was to analyze and optimize the Apache Commons-Net library across multiple dimensions:

- Ensured buildability locally and in CI/CD pipelines.
- Conducted software quality analysis with **SonarCloud**, categorizing and fixing issues.
- Created a **Docker image** for easy orchestration and testing.
- Analyzed **code coverage** with **Jacoco** and performed **mutation testing** using **PiTest**.
- Implemented **JMH performance tests** for critical components.
- Used **automated test generation** to improve coverage.
- Conducted **security analysis** with **OWASP FindSecBugs** and **OWASP DC**.

The focus was on prioritizing key areas, improving code quality, and ensuring software dependability.

THE MAIN SUBJECT OF THE ANALYSIS

What is Apache Commons-Net?

APACHE COMMONS-NET

Apache Commons-Net is a library that provides a wide range of utilities for network-related tasks.

- It supports various protocols such as **FTP, Telnet, POP3, IMAP, and SMTP**.
- Designed for developers needing **robust and reusable network functionality** in their Java applications.
- It offers an easy-to-use API for implementing network communication and file transfers.
- **Open-source** and part of the **Apache Commons** project, ensuring high quality and community support.

SOFTWARE QUALITY ANALYSIS WITH SONARCLOUD

Issues Fixing and Code Refactoring



SONARCLOUD OVERALL ANALYSIS

During SonarCloud analysis, a total of **1.3k issues** were found, categorized as in the table.

Only the high-level severity issues were addressed during this project, due to evident time and effort needed to fix them all.

Issue	High	Medium	Low
Security	2	0	0
Reliability	0	21	2
Maintainability	124	551	587

SONARCLOUD OVERALL ANALYSIS

2 high-level security issues were identified during the analysis, both categorized as "Enable server certificate validation on this SSL/TLS connection".

The solution to address both the identified security issues involve the implementation of a default **TrustManager**.

Issue	High	Medium	Low
Security	2	0	0
Reliability	0	21	2
Maintainability	124	551	587

SONARCLOUD OVERALL ANALYSIS

124 high-level maintainability **issues** were identified during the analysis.

These issues, commonly referred to as “**code smell**” are divided into multiple categories.

TAG	ACTION	# OF ISSUES
brain-overload	Refactor method to reduce its Cognitive Complexity	41
tests	Add at least one assertion to the test case	34
convention	Rename the constant name to match the regular expression	19
suspicious	Add a nested comment explaining why the method is empty, throw an UnsupportedOperationException or complete the implementation.	14
cwe	Add a default case to the switch	5
design	Define a constant instead of duplicating the literal	4
confusing	Rename field to prevent any misunderstanding	2
pitfall	Remove the non-escaped \u0007 character from the literal	2
serialization	Make * transient or serializable	2
multi-threading	Remove the call to "wait" or move it into a "while" loop	1

SONARCLOUD OVERALL ANALYSIS

The fixes and refactoring process encompassed a total of 126 issues, including 124 maintainability errors and 2 security-related vulnerabilities.

Of these, 123 issues were successfully addressed, resulting in a resolution rate of 98%. All 2 security issues were fully resolved, achieving a 100% resolution rate in this category. However, 3 errors remain unresolved, representing 2% of the total issues.

Category	High-level Issues	Fixed	Unfixed
Security	2	2 (100%)	0 (0%)
Maintainability	124	121 (98%)	3 (2%)
Total	126	123 (98%)	3 (2%)

CODE COVERAGE WITH JACOCO – CODE COVERAGE IMPROVEMENTS
WITH RANDOOP AND GITHUB COPILOT

Code Coverage Analysis and Improvements

Apache Commons Net

Element	Missed Instructions	Cov.	Missed Branches	Cov.
org.apache.commons.net		36%		24%
org.apache.commons.net.bsd		7%		0%
org.apache.commons.net.chargen		56%		n/a
org.apache.commons.net.daytime		64%		n/a
org.apache.commons.net.discard		62%		n/a
org.apache.commons.net.echo		47%		n/a
org.apache.commons.net.finger		38%		100%
org.apache.commons.net.ftp		49%		34%
org.apache.commons.net.ftp.parser		89%		77%
org.apache.commons.net.imap		25%		21%
org.apache.commons.net.io		50%		55%
org.apache.commons.net.nntp		0%		1%
org.apache.commons.net.ntp		81%		61%
org.apache.commons.net.pop3		0%		0%
org.apache.commons.net.smtp		8%		8%
org.apache.commons.net.telnet		81%		57%
org.apache.commons.net.tftp		80%		65%
org.apache.commons.net.time		72%		n/a
org.apache.commons.net.util		89%		84%
org.apache.commons.net.whois		37%		n/a
Total	15.663 of 32.632	52%	1.758 of 2.969	41%

JACOCO CODE COVERAGE

As can be observed, the code coverage is not significantly high, reaching an overall average of **52%**.

It is also important to highlight several evident and critical issues:

- Uncovered portions of the code may potentially hide malfunctions or errors that remain undetected due to these limitations;
- POP3** and **NNTP** packages show a coverage rate of 0%.

Apache Commons Net

Element	Missed Instructions	Cov.	Missed Branches	Cov.
org.apache.commons.net		49%		30%
org.apache.commons.net.bsd		9%		2%
org.apache.commons.net.chargen		56%		n/a
org.apache.commons.net.daytime		64%		n/a
org.apache.commons.net.discard		62%		n/a
org.apache.commons.net.echo		47%		n/a
org.apache.commons.net.finger		38%		100%
org.apache.commons.net.ftp		59%		43%
org.apache.commons.net.ftp.parser		89%		78%
org.apache.commons.net.imap		60%		35%
org.apache.commons.net.io		55%		55%
org.apache.commons.net.nntp		9%		4%
org.apache.commons.net.ntp		83%		69%
org.apache.commons.net.pop3		24%		14%
org.apache.commons.net.smtp		31%		18%
org.apache.commons.net.telnet		81%		58%
org.apache.commons.net.tftp		80%		66%
org.apache.commons.net.time		75%		n/a
org.apache.commons.net.util		90%		84%
org.apache.commons.net.whois		37%		n/a
Total	13.106 of 33.049	60%	1.596 of 1.359	46%

JACOCO+RANDOOP CODE COVERAGE

To improve coverage, **Randoop** has been used to generate additional test classes automatically.

Subsequently, a new coverage analysis was performed using JaCoCo, revealing significant improvements in the results, reaching an overall average of **60%**.

Apache Commons Net

Element	Missed Instructions	Cov.	Missed Branches	Cov.
org.apache.commons.net		65%		48%
org.apache.commons.net.bsd		18%		17%
org.apache.commons.net.chargen		71%		n/a
org.apache.commons.net.daytime		64%		n/a
org.apache.commons.net.discard		100%		n/a
org.apache.commons.net.echo		50%		n/a
org.apache.commons.net.finger		87%		100%
org.apache.commons.net.ftp		59%		43%
org.apache.commons.net.ftp.parser		90%		78%
org.apache.commons.net.imap		60%		35%
org.apache.commons.net.io		81%		68%
org.apache.commons.net.nttp		32%		22%
org.apache.commons.net.ntp		86%		65%
org.apache.commons.net.pop3		23%		7%
org.apache.commons.net.smtp		28%		15%
org.apache.commons.net.telnet		84%		60%
org.apache.commons.net.tftp		82%		68%
org.apache.commons.net.time		72%		n/a
org.apache.commons.net.util		91%		86%
org.apache.commons.net.whois		100%		n/a
Total	11.482 of 33.127	65%	1.483 of 2.961	50%

JACOCO+RANDOOP+COPILOT CODE COVERAGE

To further optimize code coverage, test cases generated by Randoop were supplemented with additional test cases created using **GitHub Copilot**, reaching an overall average of **65%**.

While some packages show unchanged coverage between Randoop and Copilot, others experience a slight decrease of a few percentage points, possibly due to slight variations in percentages and approximation effects but also due to Copilot changes in test classes.

Package	JaCoCo Instructions Cov.		
	Built-in Tests		
		+Randoop	+Copilot
org.apache.commons.net	36%	49%	65%
org.apache.commons.net.bsd	7%	9%	18%
org.apache.commons.net.chargen	56%	56%	71%
org.apache.commons.net.daytime	64%	64%	64%
org.apache.commons.net.discard	62%	62%	100%
org.apache.commons.net.echo	47%	47%	50%
org.apache.commons.net.finger	38%	38%	87%
org.apache.commons.net.ftp	49%	59%	59%
org.apache.commons.net.ftp.parser	89%	89%	90%
org.apache.commons.net.imap	25%	60%	60%
org.apache.commons.net.io	50%	55%	81%
org.apache.commons.net.nntp	0%	9%	32%
org.apache.commons.net.ntp	81%	83%	86%
org.apache.commons.net.pop3	0%	24%	23%
org.apache.commons.net.smtp	8%	31%	28%
org.apache.commons.net.telnet	81%	81%	84%
org.apache.commons.net.tftp	80%	80%	82%
org.apache.commons.net.time	72%	75%	72%
org.apache.commons.net.util	89%	90%	91%
org.apache.commons.net.whois	37%	37%	100%
OVERALL TOTAL	52%	60%	65%

COVERAGE COMPARISON

The overall recap highlights a notable improvement in the project's metrics. Specifically (compared to the previous):

- **Green:** Indicates an **increase** in code coverage.
- **Yellow:** Denotes **no change** in code coverage.
- **Red:** Reflects a **decrease** in code coverage.

This visual representation provides a clear view of the progression and areas requiring further attention.

ANALYZING PROJECT TEST CASES WITH PITEST

Mutation Test



Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
115	51% 3484/6767	40% 1736/4341	79% 1736/2206

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
org.apache.commons.net.bsd	3	13% 13/97	4% 3/68	50% 3/6
org.apache.commons.net.chargen	2	74% 14/19	22% 2/9	40% 2/5
org.apache.commons.net.daytime	2	85% 11/13	33% 2/6	100% 2/2
org.apache.commons.net.discard	2	82% 14/17	33% 3/9	38% 3/8
org.apache.commons.net.echo	2	80% 12/15	33% 3/9	50% 3/6
org.apache.commons.net.finger	1	82% 14/17	10% 1/10	50% 1/2
org.apache.commons.net.ftp	18	53% 945/1800	32% 335/1053	70% 335/482
org.apache.commons.net.ftp.parser	17	89% 789/889	74% 404/549	84% 404/480
org.apache.commons.net.imap	6	31% 130/423	18% 46/249	90% 46/51
org.apache.commons.net.nntp	13	1% 11/892	0% 2/590	40% 2/5
org.apache.commons.net.ntp	5	80% 269/337	71% 229/322	84% 229/272
org.apache.commons.net.pop3	6	0% 0/312	0% 0/166	100% 0/0
org.apache.commons.net.smtp	8	8% 31/366	4% 10/230	91% 10/11
org.apache.commons.net.telnet	10	74% 602/810	55% 297/538	72% 297/413
org.apache.commons.net.tftp	9	83% 346/417	73% 164/225	86% 164/190
org.apache.commons.net.time	2	91% 21/23	41% 13/32	59% 13/22
org.apache.commons.net.util	8	82% 259/315	81% 221/273	88% 221/250
org.apache.commons.net.whois	1	60% 3/5	33% 1/3	100% 1/1

Report generated by [PIT](#) 1.17.1

Enhanced functionality available at [arcmutate.com](#)

PITEST MUTATION COVERAGE

Data on the report, for mutated classes only, can be summarized as follows:

- Line Coverage is **51%** (3484 lines on 6767);
- Mutation Coverage is **40%** (killed 1736 mutations on 4341);
- Test Strength is **79%**.

Packages that have overall achieved best results across the three parameters are: **FTP.Parser**, **Util** and **NTP**.

Packages that have overall achieved worst results across the three parameters are: **POP3**, **NNTP** and **SMTP**.

ANALYZING COMPONENTS WITH JMH

Performance Testing



Operation	Throughput (ops/s)	Error (%)	Confidence 95%
Bulk Data Transfer	1,104,954,895 ops/s	~0.6%	[1,098M, 1,111M]
Connect	7,792 ops/s	~1.7%	[7,655, 7,930]
Disconnect	1,218,119,954 ops/s	~0.8%	[1,208M, 1,227M]
IsConnected	1,191,130,781 ops/s	~2.9%	[1,157M, 1,226M]
Read Response	1,170,784,231 ops/s	~3.8%	[1,126M, 1,215M]
Send Command	1,177,434,287 ops/s	~3.6%	[1,134M, 1,220M]

JMH ANALYSIS

The class selected for analysis was **TelnetClient.java**, as it emerged during testing as one of the few with significantly longer testing durations compared to others.

The benchmarks demonstrate a good level of overall consistency. The Error values are relatively small compared to the scores, ranging from 0.6% to 3.8%. The use of 5 forks, each with 5 iterations for warmup and measurement, indicates that the results have been adequately stabilized.

ANALYZING PROJECT WITH FINDSECBUGS AND OWASPDC

Security Analysis



SpotBugs Report

Project Information

Project: Apache Commons Net

SpotBugs version: 4.8.6

Code analyzed:

- /Users/jack/git/commons-net/target/classes

Metrics

14928 lines of code analyzed, in 226 classes, in 31 packages.

Metric	Total	Density*
High Priority Warnings	1	0.07
Medium Priority Warnings	16	1.07
Total Warnings	17	1.14

(* Defects per Thousand lines of non-commenting source statements)

Contents

- [Bad practice Warnings](#)
- [Security Warnings](#)
- [Dodgy code Warnings](#)
- [Details](#)

Summary

Warning Type	Number
Bad practice Warnings	11
Security Warnings	5
Dodgy code Warnings	1
Total	17

FINDSECBUGS ANALYSIS

The identified security issues have been classified into the following categories:

- Bad Practice Warnings
- Security Warnings
- Dodgy Code Warnings

All issues have been addressed and resolved, except for **Bad Practice Warnings**, which turned out to be false positives.



DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by this tool. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis or the results of the analysis.

[How to read the report](#) | [Suppressing false positives](#) | Getting Help: [github issues](#)

 [Sponsor](#)

Project: Apache Commons Net

commons-net:commons-net:3.12.0-SNAPSHOT

Scan Information ([show all](#)):

- *dependency-check version*: 11.1.1
- *Report Generated On*: Fri, 27 Dec 2024 19:07:12 +0100
- *Dependencies Scanned*: 11 (9 unique)
- *Vulnerable Dependencies*: 0
- *Vulnerabilities Found*: 0
- *Vulnerabilities Suppressed*: 0
- ...

Summary

Display: [Showing Vulnerable Dependencies \(click to show all\)](#)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
------------	-------------------	---------	------------------	-----------	------------	----------------

Dependencies (vulnerable)

This report contains data retrieved from the [National Vulnerability Database](#).

This report may contain data retrieved from the [CISA Known Exploited Vulnerability Catalog](#).

This report may contain data retrieved from the [Github Advisory Database \(via NPM Audit API\)](#).

This report may contain data retrieved from [RetireJS](#).

This report may contain data retrieved from the [Sonatype OSS Index](#).

OWASPDC ANALYSIS

An analysis was conducted using OWASP Dependency-Check to assess vulnerabilities. A total of 11 dependencies were examined, and no vulnerabilities of any kind were identified.

Thank You

MASTERS' DEGREE COURSE IN COMPUTER SCIENCE – IOT
SOFTWARE DEPENDABILITY PROJECT - AA 2024/25